Project 1: Performance Modeling and Analysis

A study of the comparison of different environments across various neural network models

COMS 6998: Applied Machine Learning for the Cloud
Columbia University
March 27, 2026
Madhav Rajkondawar

1. Experiment Design

Objective:
The goal of this project is to see how different neural network architectures actually perform when you move them across different hardware. I tested three models, namely ResNet18, ResNet50, and VGG16, against three different compute setups: a local Macbook Pro M1 Max CPU, an NVIDIA T4, and an NVIDIA L4. I compared the throughput results in order to analyze the output as well as what we learn from it.

Hypothesis:
My hypothesis is that the NVIDIA L4 will produce much better throughput performance than the NVIDIA T4 due to its high peak FLOP/s and memory bandwidth and the Macbook CPU will be 10-100 times below the performance of both NVIDA GPUs due to no ability to execute parallely. I think that VGG16 will achieve high GFLOP/s due to its high number of FLOPs for each forward pass even though it has low arithmetic intensity. Finally, I expect each of the models to be memory-bandwidth-limited on the GPUs because arithmetic intensity will be below the hardware ridge points.

Environments:
There were 3 compute environments that I used here which represented a large spectrum ranging from the CPU to the modern GPU:

Environment	Hardware	Peak GFLOP/s	Peak Bandwidth	Memory
Mac CPU	Apple M1 Max (ARM)	~800	50 GB/s	32 GB Unified
Colab T4	NVIDIA Tesla T4	8,100	300 GB/s	15.4 GB VRAM
Colab L4	NVIDIA L4	31,300	864 GB/s	22.5 GB VRAM

Models Used:
Below we have the specific CNN models that were used to signify the different architectural differences and complexities:

Model	Parameters	FLOPs/sample	Architecture
ResNet18	11.7M	1.82 GFLOPs	18-layer residual network
ResNet50	25.6M	4.10 GFLOPs	50-layer residual network
VGG16	138.4M	15.5 GFLOPs	16-layer sequential CNN

2. Complexity Estimation

Arithmetic Intensity (AI):
AI here is the ratio of operations that are floating points to the data movement bytes. The AI in this case was calculated as AI = FLOPs / (2 x param_bytes + input_bytes) where the parameter bytes is multiplied by 2 taking into account the weight that is being read during the gradient writing as well as the forward pass. Here are the values that we got as a result:

- ResNet18: 19.3 FLOPs/byte
- ResNet50: 19.9 FLOPs/byte
- VGG16: ~14 FLOPs/byte

As we can see all of the models above have been lying comfortably below the ridge points of all the hardware that was tested. This does confirm that all our models across all the environments are bound by the memory-bandwidth which means that the performance is mainly limited to the speed at which the data can be moved from memory to compute rather than the capacity of the raw computation.

3. Measurement

Approach:
In order to achieve the results that we found in this section, I implemented PyTorch along with CUDA-synchronized timing specifically for the GPU environments namely the T4 and L4 compute environments. Each of the measurements here made usage of 5 iterations just to start off which then led to 20 iterations that were timed utilizing time.perf_counter() with torch.cuda.synchronize() in order to make sure that we get the correct GPU measurements. I made sure to use 32 as the batch size for all of the environments to maintain consistency.

Below are the graphs showing the data for all the environments and their respective models used:

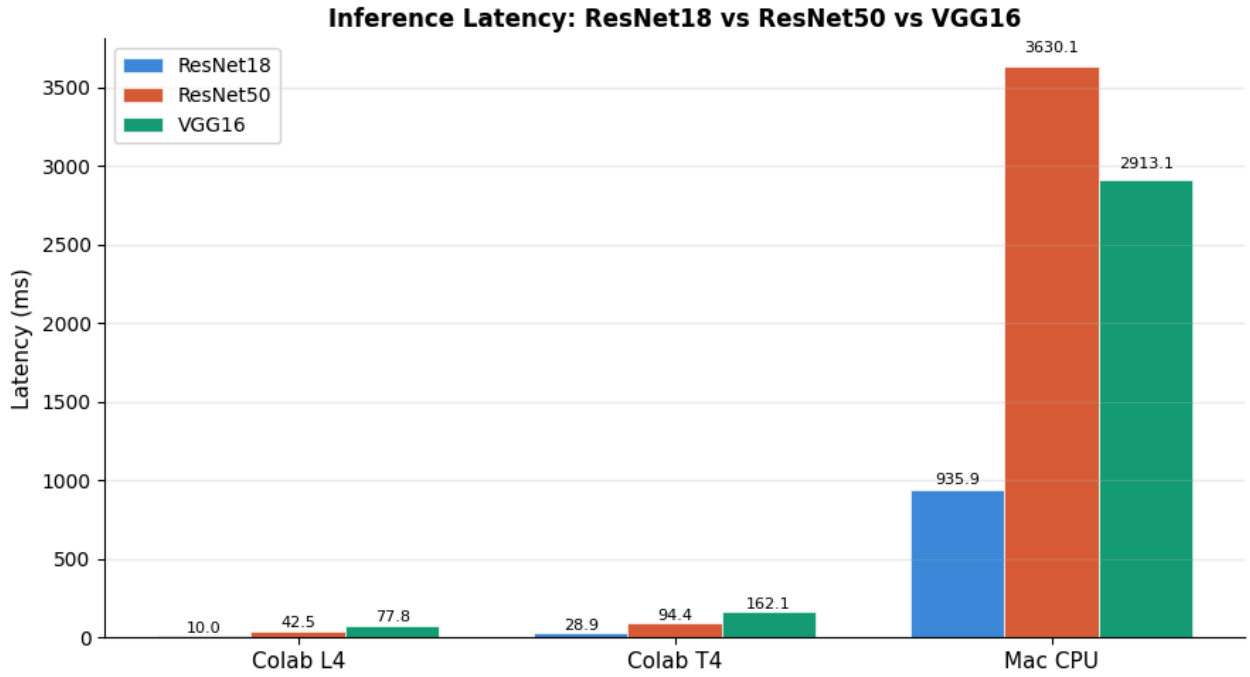

Figure 1: Inference Latency for all the models and environments

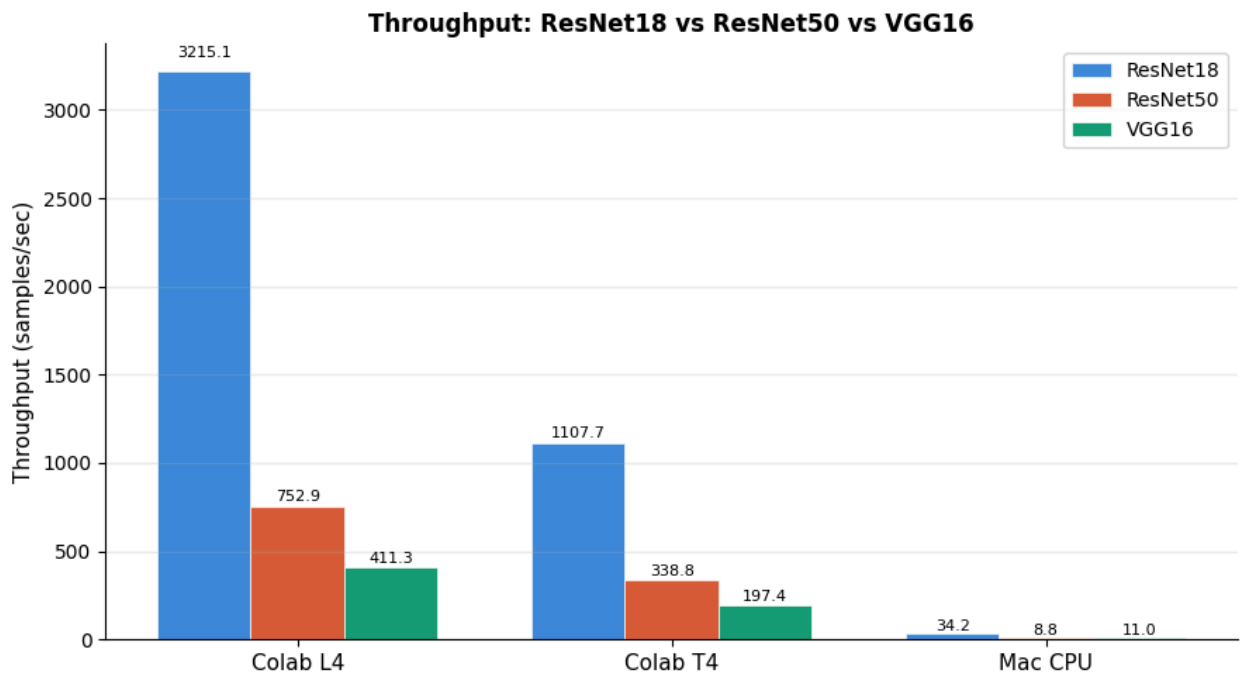

Figure 2: Throughput for all the models and environments

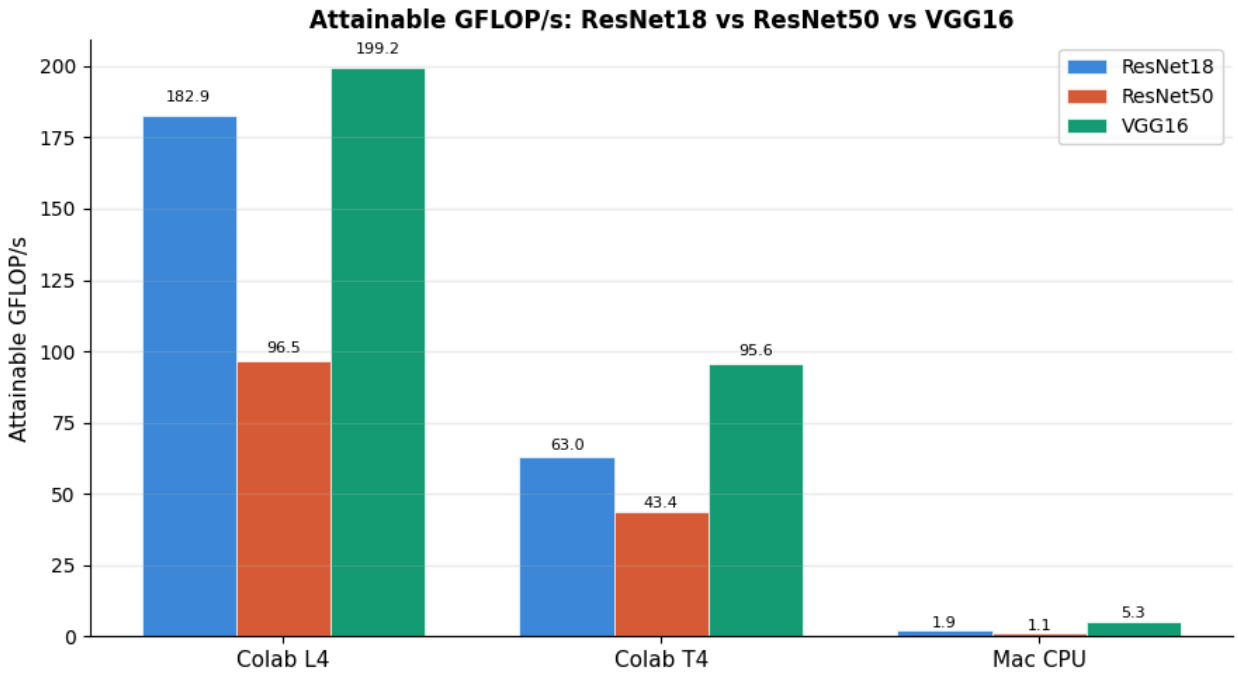

Figure 3: Attainable GFLOP/s for all the models and environments

Below are the statistics I got for each of the compute environments when I ran them on Google Colab as well as locally on my CPU:

```
Environment : mac_cpu
Device      : cpu
Platform    : macOS-26.3.1-arm64-arm-64bit-Mach-O
Processor   : arm
PyTorch     : 2.11.0

Warning: CPU runs are slow — VGG16 may take several minutes.

Profiling ResNet18...
  Params    : 11,689,512 (44.6 MB)
  FLOPs     : 1,820,000,000
  Latency   : 935.854 ± 64.35 ms
  Throughput: 34.2 samples/sec
  AI        : 19.3374 FLOPs/byte
  GFLOP/s   : 1.94

Profiling ResNet50...
  Params    : 25,557,032 (97.5 MB)
  FLOPs     : 4,100,000,000
  Latency   : 3630.119 ± 128.656 ms
  Throughput: 8.8 samples/sec
  AI        : 19.9943 FLOPs/byte
  GFLOP/s   : 1.13

Profiling VGG16...
  Params    : 138,357,544 (527.8 MB)
  FLOPs     : 15,500,000,000
  Latency   : 2913.137 ± 231.422 ms
  Throughput: 11.0 samples/sec
  AI        : 13.996 FLOPs/byte
  GFLOP/s   : 5.32
```

```
Environment : colab_l4
Device      : cuda
GPU         : NVIDIA L4
Memory      : 22.0 GB
CUDA        : 12.8

Profiling ResNet18...
  Params    : 11,689,512 (44.6 MB)
  FLOPs     : 1,820,000,000
  Latency   : 9.953 ± 0.197 ms
  Throughput: 3215.1 samples/sec
  AI        : 19.3374 FLOPs/byte
  GFLOP/s   : 182.86
  GPU Mem   : 700.09 MB

Profiling ResNet50...
  Params    : 25,557,032 (97.5 MB)
  FLOPs     : 4,100,000,000
  Latency   : 42.502 ± 0.342 ms
  Throughput: 752.9 samples/sec
  AI        : 19.9943 FLOPs/byte
  GFLOP/s   : 96.47
  GPU Mem   : 700.09 MB

Profiling VGG16...
  Params    : 138,357,544 (527.8 MB)
  FLOPs     : 15,500,000,000
  Latency   : 77.811 ± 0.99 ms
  Throughput: 411.3 samples/sec
  AI        : 13.996 FLOPs/byte
  GFLOP/s   : 199.2
  GPU Mem   : 700.09 MB
```

```
Environment : colab_t4
Device      : cuda
GPU         : Tesla T4
Memory      : 14.6 GB
CUDA        : 12.8

Profiling ResNet18...
  Params    : 11,689,512 (44.6 MB)
  FLOPs     : 1,820,000,000
  Latency   : 28.888 ± 2.677 ms
  Throughput: 1107.7 samples/sec
  AI        : 19.3374 FLOPs/byte
  GFLOP/s   : 63.0
  GPU Mem   : 699.21 MB

Profiling ResNet50...
  Params    : 25,557,032 (97.5 MB)
  FLOPs     : 4,100,000,000
  Latency   : 94.446 ± 0.659 ms
  Throughput: 338.8 samples/sec
  AI        : 19.9943 FLOPs/byte
  GFLOP/s   : 43.41
  GPU Mem   : 699.21 MB

Profiling VGG16...
  Params    : 138,357,544 (527.8 MB)
  FLOPs     : 15,500,000,000
  Latency   : 162.075 ± 0.917 ms
  Throughput: 197.4 samples/sec
  AI        : 13.996 FLOPs/byte
  GFLOP/s   : 95.63
  GPU Mem   : 699.21 MB
```

Below I have summarized the data into a neat table comparing all the different models for each of the environments:

Environment	**Model**	**Std (ms)**	**Throughput (per second)**	**GFLOP/s**	**Latency**
Mac CPU	ResNet18	64.4	34.2	1.94	935.9
Mac CPU	ResNet50	128.7	8.8	1.13	3630.1
Mac CPU	VGG16	231.4	11.0	5.32	2913.1
Colab T4	ResNet18	2.7	1107.7	63.0	28.9
Colab T4	ResNet50	0.7	338.8	43.41	94.4
Colab T4	VGG16	0.9	197.4	95.63	162.1
Colab L4	ResNet18	0.2	3215.1	182.9	10.0
Colab L4	ResNet50	0.3	752.9	96.47	42.5
Colab L4	VGG16	0.99	411.3	199.2	77.8

```
(venv}) MacBookPro:Project 1 - Performance modeling and analysis mrajk$ python plot_roofline.py

Env          Model            AI    GFLOP/s       ms    samples/s
-------------------------------------------------------------
colab_l4     ResNet18     19.337      182.9     10.0       3215.1
colab_l4     ResNet50     19.994       96.5     42.5        752.9
colab_l4     VGG16        13.996      199.2     77.8        411.3
colab_t4     ResNet18     19.337       63.0     28.9       1107.7
colab_t4     ResNet50     19.994       43.4     94.4        338.8
colab_t4     VGG16        13.996       95.6    162.1        197.4
mac_cpu      ResNet18     19.337        1.9    935.9         34.2
mac_cpu      ResNet50     19.994        1.1   3630.1          8.8
mac_cpu      VGG16        13.996        5.3   2913.1         11.0
```

Below I have the speedup versus the CPU baseline comparison across all the models and environments tested:

Model	**L4 vs CPU**	**T4 vs CPU**	**L4 vs T4**
ResNet18	93.6x	32.4x	2.9x
ResNet50	85.4x	38.5x	2.2x
VGG16	37.4x	18.0x	2.1x

4. Roofline Modeling

Roofline Model Building:
The roofline model specifically defines two performance limits:
Bandwidth limit: AI × memory bandwidth
Compute limit: peak GFLOP/s

The ridge point separates these two regions:

Ridge Point = Peak GFLOP/s / Peak Bandwidth

For each platform:
CPU: ~16 FLOPs/byte
T4: ~27 FLOPs/byte
L4: ~36.2 FLOPs/byte

Charts Generated:
The plots below help visualize where each model lies relative to hardware limits. Below are the charts that I got through the code that I wrote showing the roofline modeling for individual models and their environments leading into a final graph with all of them:

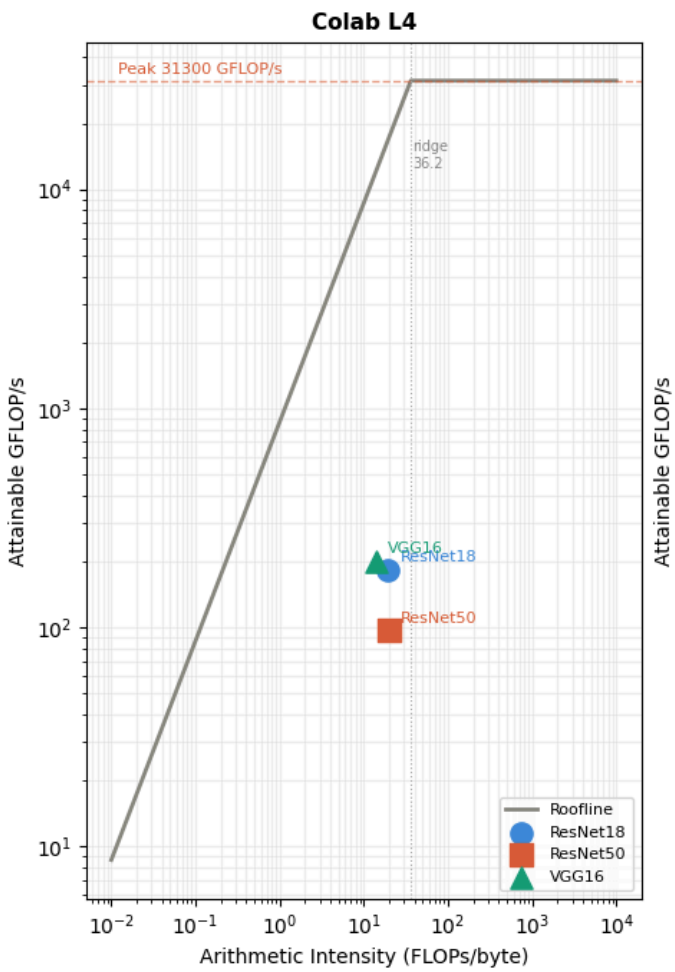

Figure 4: Colab L4

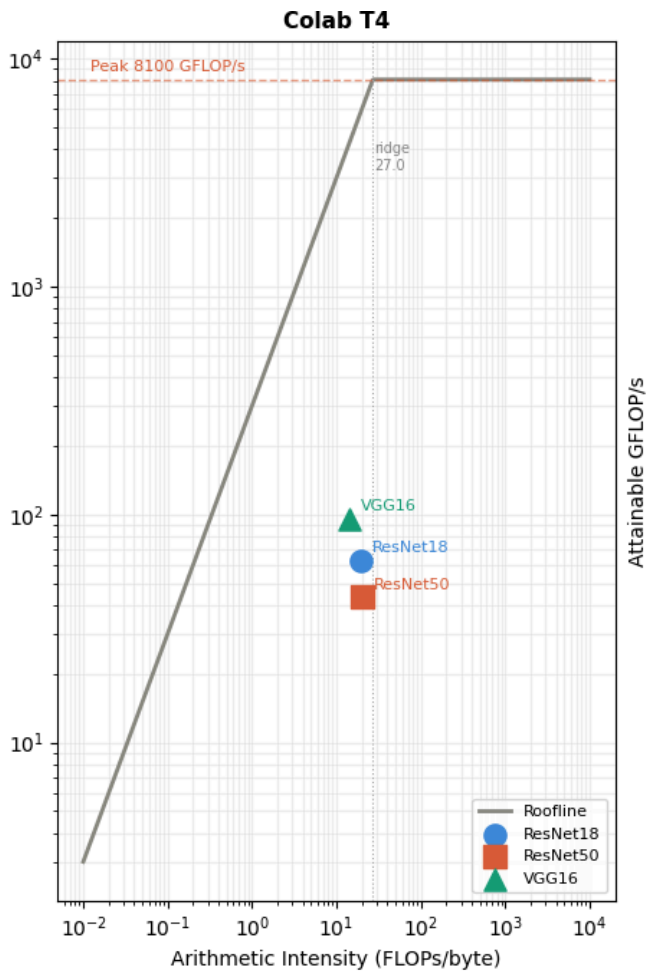

Figure 5: Colab T4

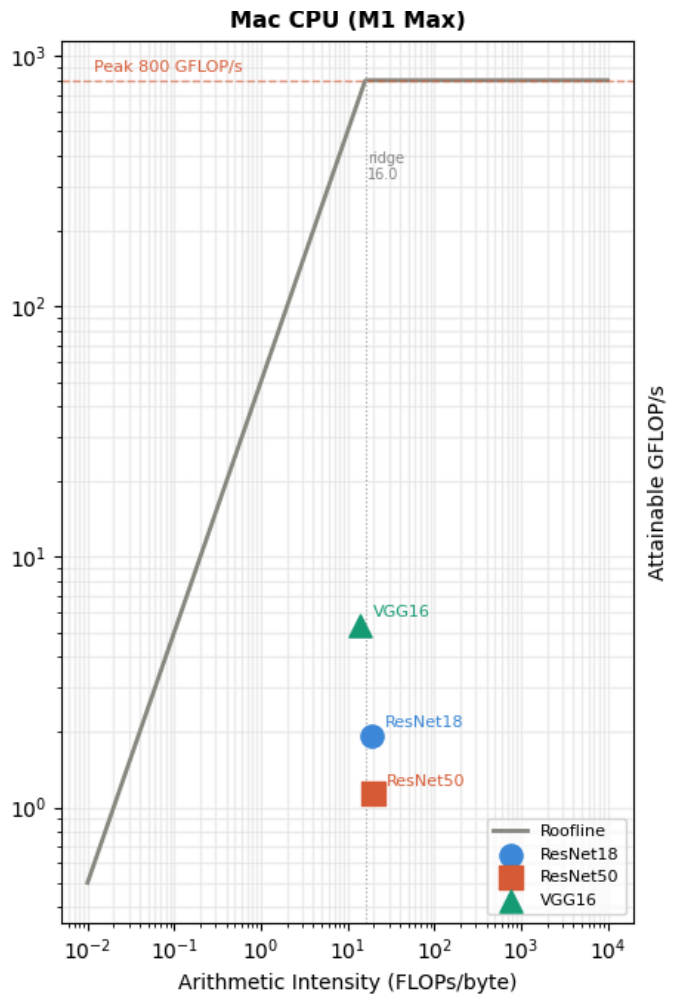

Figure 6: Mac CPU

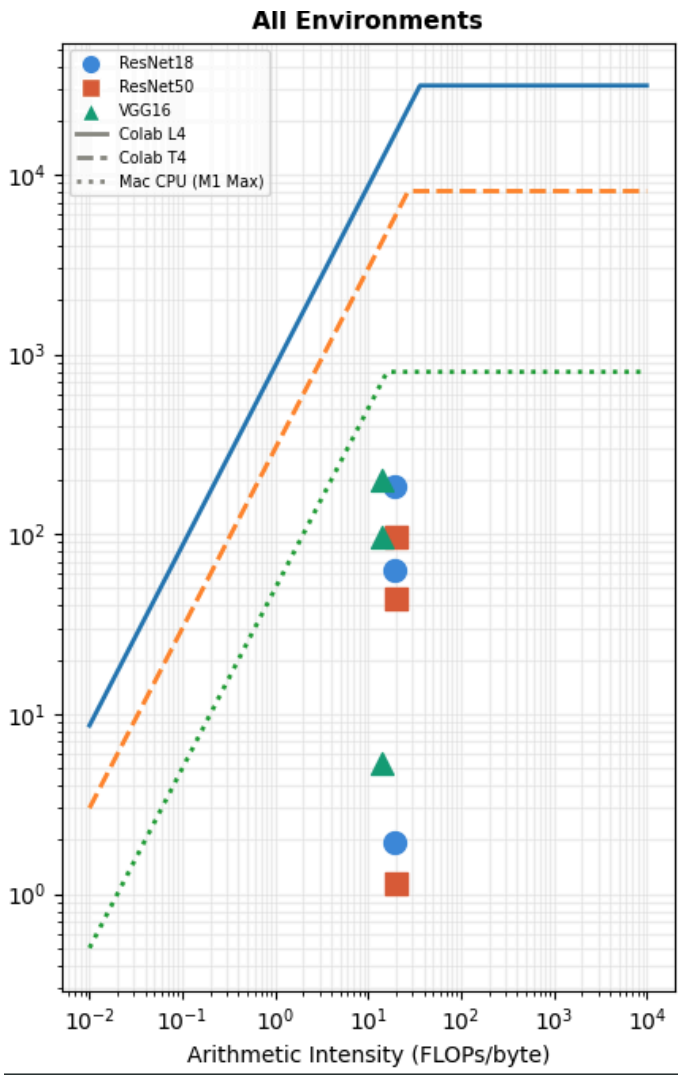

Figure 7: All environments

Model Positioning:

From the data, we can see that all the models fall below the ridge point on GPU platforms, which places them in the memory-bound region. This means that increasing compute alone would not significantly improve performance and we see that the memory bandwidth is the main bottleneck. Talking about the CPU, ResNet models appear slightly above the ridge point, but this does not translate to good performance due to the CPU's limited compute capacity.

5. Analysis

Performance impact with environment:
A notable discovery reveals a significant disparity between CPU and GPU performance. The L4 GPU demonstrates a throughput that is 37-94 times greater than that of the Mac CPU across all evaluated models. This discrepancy arises from key architectural distinctions:

- The M1 Max CPU is equipped with approximately 10 cores that process vectorized instructions one at a time. In contrast, GPU systems feature thousands of CUDA cores operating concurrently, with the T4 containing 2,560 CUDA cores and the L4 boasting 7,424 cores.
- The memory bandwidth of GPUs is 6-17 times greater than that of CPUs, with the T4 achieving 300 GB/s compared to around 50 GB/s for the CPU. Given that all models are restricted by memory bandwidth, performance is closely tied to this metric.
- GPUs are specifically designed for the matrix multiplication tasks that are prevalent in CNN inference, utilizing dedicated tensor cores to enhance these processes.

The L4 provides a performance enhancement of roughly 2 to 3 times over the T4 across all models, illustrating the L4's generational improvements: it offers a peak GFLOP/s rate that is 3.9 times higher and memory bandwidth that is 2.9 times greater. The overall speed increase is not proportional to the raw FLOP ratio due to the memory bandwidth acting as the limiting factor, rather than computational capability.

Discussion of Model Architecture:
Here's what stands out when you compare these models: VGG16 hits the highest absolute GFLOP/s on GPUs, particularly 95.6 on T4 and 199.2 on L4. That's impressive, especially given its low arithmetic intensity (just 13.99 FLOPs per byte). Sounds odd, right? However, it makes sense if you look at how many FLOPs it burns through. VGG16 pulls off about 8.5 times more floating-point operations per inference where we have 15.5 GFLOPs per sample, while ResNet18 clocks in at only 1.82 GFLOPs.

Even though VGG16 is memory-bound, the sheer volume of computation keeps the GPU busy longer. That's why its measured GFLOP/s end up so high. Flip the comparison, and you see ResNet50 actually posts lower GFLOP/s than ResNet18, showing particularly 43.4 versus 63.0 on T4, and 96.5 versus 182.9 on L4. This is noted despite ResNet50 doubling the FLOPs per sample (2.25 times more).

This is where things get interesting: ResNet50 takes way longer to run on the T4, particularly 94.4ms compared to ResNet18's snappy 28.9ms, even though it's only doing about twice as many operations. This happens because of GPU occupancy and parallelism. ResNet18 has wider layers for its depth, so it can push more parallel operations per layer. ResNet50, on the other hand, uses bottleneck blocks and keeps tossing out short-lived kernels that never fully saturate those GPU warps. Additionally, ResNet50 has to launch 50 kernels in sequence, versus only 18 kernel launches for ResNet18. That overhead adds up fast and can affect performance significantly.

ResNet18 really takes advantage of faster hardware. Moving from CPU to L4, it's 93.6 times quicker, while VGG16 only gets a 37.4 times speedup. The secret? ResNet18's small parameter size being just 44.6 MB, which fits neatly into L4's 48MB L2 cache. So DRAM traffic drops, speeding things up. VGG16, though, is stuck with a huge weight matrix (527.8 MB), way bigger than any GPU cache out there. That means its weights constantly get evicted, and DRAM bandwidth ends up being the bottleneck, no matter which GPU you're using.

Interpretation of the roofline data:
All the models fall way below the roofline ceiling on every platform, which basically means there's plenty of untapped performance right now. The gap between what you measure and the roofline isn't just a number but it's the result of several inefficiencies:

- Kernel launch overhead: Each layer needs its own GPU kernel launch, tacking on microseconds every time. Stack up enough layers, and that overhead adds up fast.
- Memory access patterns: When batch normalization and residual connections reach for non-contiguous chunks of memory, you end up wasting memory bandwidth.
- GPU underutilization at batch size 32: With just 32 samples per batch, smaller models like ResNet18 don't throw enough work at the L4's 7,424 CUDA cores. Most of them just sit idle. If you crank up the batch size, you keep the GPU way busier and start closing that gap leading to a more efficient utilization of space.

Here is what the headroom really looks like: ResNet18 hits just 182.9 GFLOP/s on the L4. The roofline says you could be looking at about 5,802 GFLOP/s (taking 300 GB/s memory bandwidth times 19.34 AI). So you're using only 3.2% of what's theoretically possible, which is a massive gap. That tells you the big problem for small models on new GPUs isn't model complexity, but it's batch size.

There are two tweaks that would get models much closer to the roofline. First, switching to FP16 mixed precision (using torch.autocast) doubles arithmetic intensity: FP16 cuts memory usage for parameters in half, but the number of FLOPs stays the same, so AI jumps from about 19 to 38 FLOPs/byte. That push might get you out of the memory-bound zone and into compute-bound territory. Second, bumping the batch size up from 32 to 256 or even 512 spreads out kernel launch costs across more samples, which seriously boosts GPU occupancy and throughput. Both tricks are standard practice in production.

Take a look at the combined roofline chart (Figure 7). You'll see all the model dots float upward as you go from CPU to T4 to L4, but they stick around the same spot horizontally (since AI doesn't depend on the hardware). This shows that better hardware directly ramps up achievable GFLOP/s, but doesn't change the fact that these workloads are still fundamentally memory-bound.

Effect on deployment:
For real life application, here is how the different models lay before us:

- If the application can't afford to wait and needs real-time updates, then you really need the L4 GPU. It crunches through ResNet18 at 10ms per image. That's good enough for 100 frames per second on video. Compare that to the CPU, which takes 936ms per image. In other words, the CPU can't even play in this arena.
- For batch jobs where you just want to maximize throughput, the story's the same. The L4 can handle 3,215 ResNet18 samples every second, while the CPU barely gets through 34. That's a 94x leap, and it translates straight to lower cloud costs.
- Model choice matters a lot here. On the L4, ResNet18 does 3,215 samples per second, while ResNet50 only hits 752. You get 4.3 times the throughput with ResNet18, and you only lose a little in accuracy, specifically at least for ImageNet. Unless you absolutely have to squeeze out every bit of top-1 accuracy, ResNet18 is the way to go.
- If you want to push things even further, try bumping up your batch size, switching to mixed-precision (FP16) to make bandwidth go twice as far, and using fused kernels like torch.compile to shave off kernel launch overhead. That's how you move closer to the hardware's limits.

Future and Limitations to address:

While the results are clear, there are a few things to keep in mind regarding the accuracy and scope of this test:

- Underestimated Memory Traffic: My Arithmetic Intensity (AI) calculation is essentially a lower bound. I only counted parameters and input bytes, but I didn't track intermediate activation memory traffic or temporary buffers. If I used hardware counters like Nsight Compute, the measured DRAM traffic would likely be 2x-5x higher. This would actually push the dots further to the left on the roofline chart, making the models even more memory-bound than they already look.
- Colab Variability: Using shared cloud GPUs in Google Colab isn't always perfect. I noticed some noise in the timing, especially with ResNet18 on the T4, where the standard deviation was 2.7ms (about 9.3% of the mean). This is likely just scheduling noise from the shared environment.
- Synthetic Data vs. Real Images: I used synthetic random tensors instead of real ImageNet images to keep things simple. The downside is that real images have spatial correlations that help with cache reuse. Random data lacks these patterns, so the cache behavior here might be slightly worse than a real production workload.
- Inference Only: This study was strictly about the forward pass. Training is a totally different beast because you have to deal with gradient computation and optimizer states, which would change the roofline positions significantly.

What's Next?

If I had more time, I'd use Nsight Compute to get precise hardware measurements. I'd also love to test the impact of FP16 mixed precision and much larger batch sizes to see just how close I can push these models to the hardware's theoretical "roof".

6. Conclusion

In this study, I profiled three common CNN architectures: ResNet18, ResNet50, and VGG16 across a local Mac M1 Max, an NVIDIA T4, and an NVIDIA L4. By applying the roofline model, I was able to move past simple "stopwatch" timing and actually see the hardware-level constraints limiting each model.

Final Takeaways from this study:

- The Memory Wall is Real: Every model I tested on the GPUs was bound by the memory-bandwidth. Their arithmetic intensities (13.99–19.99 FLOPs/byte) were all too low to reach the "ridge point" of the T4 (27) or the L4 (36.2), meaning the hardware was stuck waiting for data rather than being limited by math speed.
- Generational Leaps: The L4 is the clear winner for production, delivering a 2.1x–2.9x speedup over the T4 and an incredible 37x–94x jump over the Mac CPU baseline.
- The VGG16 Paradox: VGG16 actually hit the highest absolute GFLOP/s despite having the lowest efficiency (AI). Due to it having such a massive FLOP count, it keeps the GPU cores saturated longer than the lighter ResNet models.
- Top Pick for Deployment: For any real-world app where speed matters, ResNet18 is the best choice. It provided the highest throughput per parameter, making it the most bang-for-your-buck model tested.
- Optimization Potential: There is still plenty of room to grow. All my results sat below the theoretical "roof," suggesting that using larger batch sizes, mixed precision (FP16), or kernel fusion could push performance even higher.

Ultimately, the roofline model was a great framework for this project. It didn't just tell me that the L4 was faster, but explained why (bandwidth) and showed exactly how much performance I was leaving on the table.